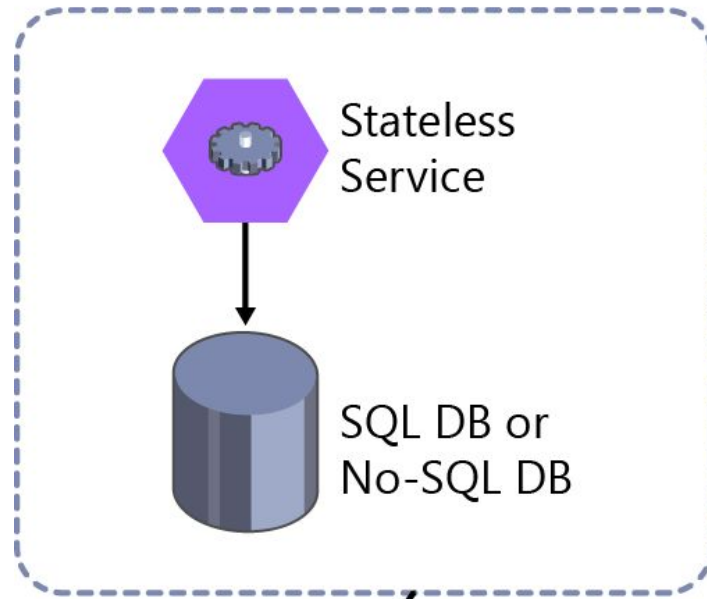




Stateless Vs Stateful

Stateless Services

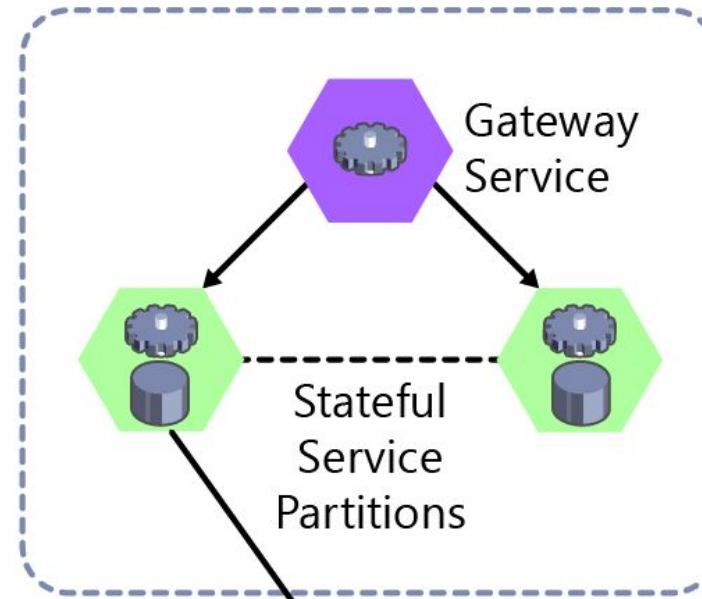
Business
Microservice A



Stateless microservices
with separate store

Stateful Services

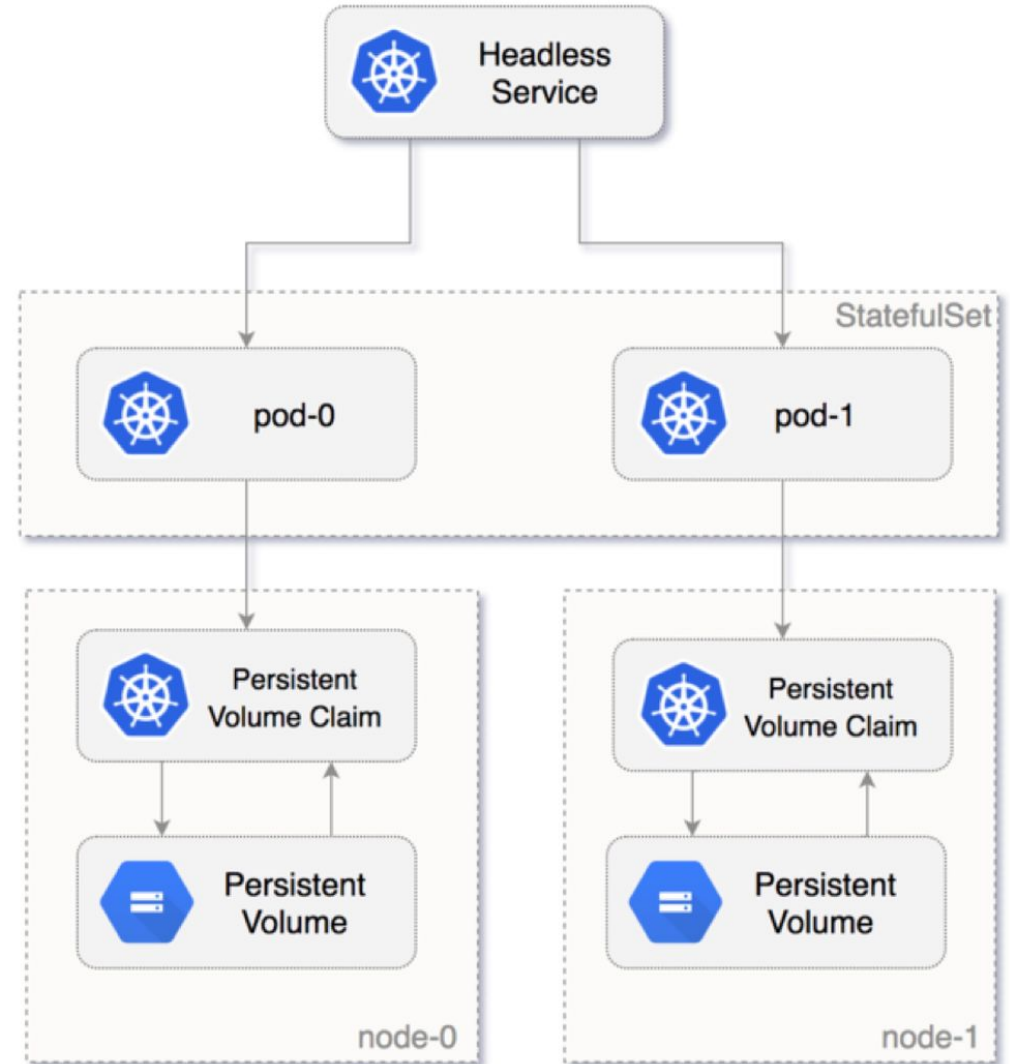
Business
Microservice B



Stateful microservice with
in-memory data. Low latency
between business logic and data

Kubernetes StatefulSet

- Kubernetes StatefulSet is a resource in Kubernetes used for managing stateful applications that require stable network identities and persistent storage. It ensures each pod has a unique identity, stable network hostname, and persistent storage. StatefulSets support ordered and parallel deployment, scaling, and rolling updates, making it easier to manage stateful applications in Kubernetes.



StatefulSet Vs Deployment

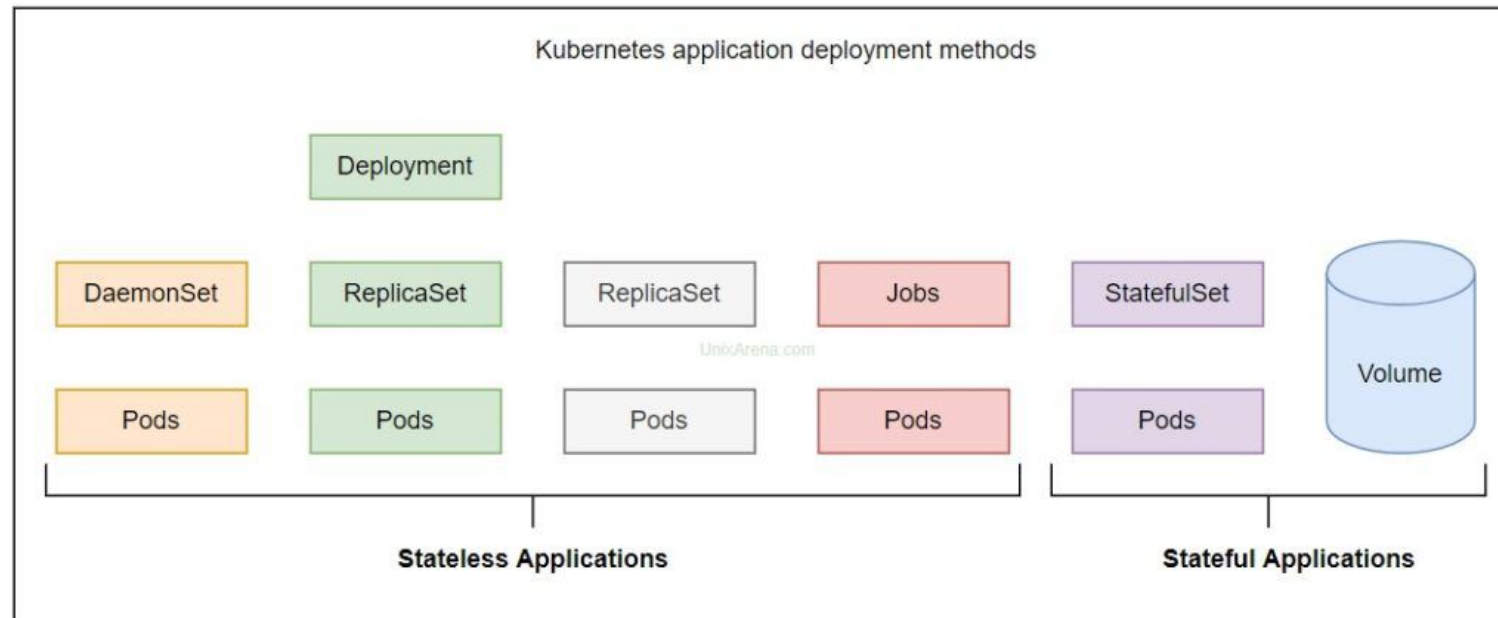
- Kubernetes StatefulSet is designed for managing stateful applications that require stable network identities and persistent storage. Key characteristics of StatefulSet include:
 - Pods are created one by one
 - Pods are deleted in reverse order
 - Sticky and predictable name is assigned to the pods
 - Different PVs are used for each pod
 - Scale is difficult

StatefulSet is suitable for applications like databases, distributed file systems, and other stateful applications that require stable identities and storage.

StatefulSet Vs Deployment

- Deployment: Deployment is designed for managing stateless applications or applications that do not require stable network identities or persistent storage. Key characteristics of Deployment include:
 - All pods are created parallelly
 - Pods are deleted randomly
 - Random name are assigned to all pods
 - Same PV is used for all pods
 - Scaling is easy

Deployment is commonly used for web servers, microservices, and other stateless applications that can be easily replicated and scaled.



StatefulSet with MongoDB

- Dynamic Local Provisioning from Rancher
 - `kubectl apply -f`
<https://raw.githubusercontent.com/rancher/local-path-provisioner/master/deploy/local-path-storage.yaml>
 - `kubectl get sc`

```
kaoleangseng — kao.leangseng@node1: ~ — ssh leangsengk90@34.126.113.232 — 140x40
kao.leangseng@node1:~$ kubectl get sc
NAME          PROVISIONER          RECLAIMPOLICY    VOLUMEBINDINGMODE    ALLOWVOLUMEEXPANSION    AGE
local-path    rancher.io/local-path Delete            WaitForFirstConsumer  false                   34h
```

StatefulSet with MongoDB

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongo
spec:
  selector:
    matchLabels:
      app: mongo
  serviceName: "mongo"
  replicas: 3
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
        - name: mongo
          image: mongo:4.0.8
          args: ["--dbpath", "/data/db"]
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              value: "admin"
            - name: MONGO_INITDB_ROOT_PASSWORD
              value: "admin"
```

```
# env level:
  command:
    - mongod
    - "--bind_ip_all"
    - "--replSet"
    - rs0
  volumeMounts:
    - name: mongo-volume
      mountPath: /data/db
volumeClaimTemplates:
  - metadata:
      name: mongo-volume
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: local-path
      resources:
        requests:
          storage: 1Gi
```

StatefulSet with MongoDB

```
# Headless Service
apiVersion: v1
kind: Service
metadata:
  name: mongo
spec:
  ports:
    - name: mongo
      port: 27017
      targetPort: 27017
  clusterIP: None
  selector:
    app: mongo
```

```
[kao.leangseng@node1:~$ kubectl get pv
NAME                                     CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS
pvc-675ea9ae-c411-4181-9268-d588dac0d5ee  1Gi       RWO           Delete          Bound
pvc-728eb8a9-0ce7-4feb-8abc-2eabb5996b47  1Gi       RWO           Delete          Bound
pvc-cf538d7c-eb83-4f4d-afee-50202d4c5aac  1Gi       RWO           Delete          Bound
```

```
[kao.leangseng@node1:~$ kubectl get pvc
NAME                                STATUS  VOLUME                                     CAPACITY
mongo-volume-mongo-0               Bound   pvc-675ea9ae-c411-4181-9268-d588dac0d5ee  1Gi
mongo-volume-mongo-1               Bound   pvc-728eb8a9-0ce7-4feb-8abc-2eabb5996b47  1Gi
mongo-volume-mongo-2               Bound   pvc-cf538d7c-eb83-4f4d-afee-50202d4c5aac  1Gi
```


StatefulSet with MongoDB

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongo-express
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongo-express
  template:
    metadata:
      labels:
        app: mongo-express
    spec:
      containers:
        - name: mongo-express
          image: mongo-express:latest
          ports:
            - containerPort: 8081
          env:
            - name: ME_CONFIG_MONGODB_URL
              value: "mongodb://mongo:27017"
            - name: ME_CONFIG_BASICAUTH_USERNAME
              value: "admin"
            - name: ME_CONFIG_BASICAUTH_PASSWORD
              value: "123"
```

```
apiVersion: v1
kind: Service
metadata:
  name: mongo-express-svc
spec:
  type: NodePort
  selector:
    app: mongo-express
  ports:
    - protocol: TCP
      port: 8081
      targetPort: 8081
      nodePort: 30003
```

StatefulSet with MongoDB

```
# kubectl exec -it pod/mongo-0 -- mongo
rs.initiate(
  {
    _id: "rs0",
    version: 1,
    members: [
      { _id: 0, host : "mongo-0.mongo:27017" },
      { _id: 1, host : "mongo-1.mongo:27017" },
      { _id: 2, host : "mongo-2.mongo:27017" }
    ]
  }
)
# exit and exec into pod again
```

```
To enable free monitoring, run the following command: db.enableFreeMonitoring()
To permanently disable this reminder, run the following command: db.disableFreeMonitoring()
---
```

```
rs0:PRIMARY> █
```

StatefulSet with MongoDB

```
# Testing in Master
# kubectl exec -it pod/mongo-0 -- mongo
use test
db.todos.insert({"title": "Do something"})
db.todos.find()
```

```
[rs0:PRIMARY> db.todos.find()
{ "_id" : ObjectId("64a88b5c6c5d2694f50e8652"), "title" : "Do something" }
```

```
# View replicated data in Slave
# kubectl exec -it pod/mongo-1 -- mongo
rs.slaveOk()
db.todos.find()
```

```
[rs0:SECONDARY> db.todos.find()
{ "_id" : ObjectId("64a88b5c6c5d2694f50e8652"), "title" : "Do something" }
```

A large, faint watermark of the ISTAD logo is centered in the background. It is a circular emblem with a blue outer ring containing the text 'INSTITUTE OF SCIENCE AND TECHNOLOGY ADVANCED DEVELOPMENT' and Khmer text. Inside the ring is a blue circle with a white atomic symbol and binary code '01010100 01000001 01000001'.

Thank you

Begin with the theory, Start with the practical